

Phase 2: Core Implementation

Generative AI Pipeline Design on Databricks

1. Design Prompts & Select Chain Components

Core Concept

In Databricks, building a robust reasoning chain requires assembling specific LangChain components: the LLM endpoint (`ChatDatabricks`), the input prompt (`ChatPromptTemplate`), and tool/output definitions. You design a prompt to securely pass system instructions and user input, ensuring the model elicits a structured format expected by downstream applications

[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/unity-catalog-tool-integration>].

How It Works on Databricks

You use the `databricks-langchain` integration package. By initializing `ChatDatabricks`, you securely connect to Databricks Model Serving (e.g., Llama 3 endpoints). You then define a `ChatPromptTemplate` containing a "system" message (guiding the behavior and output schema) and a "human" message (the query)

[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/unity-catalog-tool-integration>].

When to Use vs. Alternatives

- **Use LangChain components:** When integrating custom logic, memory, or third-party open-source orchestration in custom Python code.
- **Alternative (Databricks Supervisor API):** If you do not want to manage the prompt chain and tool execution loop yourself, deploy using the managed Supervisor API instead

[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/supervisor-api-app>].

Common Mistakes

Failing to log the execution traces. Developers often deploy chains without visibility into how the prompt was formatted or token usage. Always include `mlflow.langchain.autolog()` to automatically trace and log every step your agent takes in development and production

[cite: <https://docs.databricks.com/aws/en/agents/custom-agents/build-agents>].

Code Example & Function Explanation

```
from databricks_langchain import ChatDatabricks
from langchain_core.prompts import ChatPromptTemplate
import mlflow

# Explain function: mlflow.langchain.autolog()
# Enables MLflow Tracing for end-to-end observability, logging the chain's execution.
mlflow.langchain.autolog()

# Explain function: ChatDatabricks()
# Connects to a Databricks-hosted Foundation Model endpoint securely.
llm = ChatDatabricks(endpoint="databricks-meta-llama-3-3-70b-instruct",
temperature=0.1)

# Explain function: ChatPromptTemplate.from_messages()
# Constructs a prompt payload instructing the model to elicit a structured analytical
response.
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a B2B lead analyst. Format your response strictly as a JSON
list."),
    ("placeholder", "{chat_history}"),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])
```

2. Define & Order Tools for Multi-Stage Reasoning

Core Concept

To enable multi-stage reasoning, the model needs to query databases or trigger actions. You define these capabilities as "Tools". Databricks seamlessly governs this through Unity Catalog, allowing you to define Python or SQL logic natively as governed assets and expose them to agents

[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/create-custom-tool>].

How It Works on Databricks

You can create AI agent tools by writing Unity Catalog (UC) functions. Using the `unitycatalog-langchain[databricks]` package, you wrap the registered function with the `UCFunctionToolkit`. This ensures consistency across GenAI libraries and enforces Unity Catalog governance rules during multi-stage execution *[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/create-custom-tool>]*. For external execution or per-user auth flexibility, you can define logic directly in Python via the `@tool` decorator *[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/create-custom-tool>]*.

When to Use vs. Alternatives

- **Use UC Functions:** Best for structured data retrieval when the query is known ahead of time, ensuring enterprise-grade data governance

[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/create-custom-tool>].

- **Use MCP (Model Context Protocol):** Databricks recommends MCP servers to add tools when you need simpler integration with automatic tool discovery and built-in per-user authentication

[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/create-custom-tool>].

Common Mistakes

Failing to write clear, Google-style docstrings with explicit type hints. The Unity Catalog function toolkit relies entirely on docstrings to instruct the LLM on how and when to use the tool. Poorly documented tools lead to agent hallucinations and loop failures

[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/create-custom-tool>].

Code Example & Function Explanation

```
from databricks_langchain import UCFunctionToolkit
from langchain.agents import create_tool_calling_agent, AgentExecutor

# Explain function: UCFunctionToolkit()
# Wraps a defined Unity Catalog function so the LangChain agent understands its
signature.
# e.g., querying a commercial property database or a bakery wholesale orders table.
toolkit =
UCFunctionToolkit(function_names=["my_catalog.my_schema.get_wholesale_orders"])
tools = toolkit.tools

# Explain function: create_tool_calling_agent() & AgentExecutor()
# Binds the tools and prompt to the LLM, enabling the agent loop to execute multi-
stage reasoning.
agent = create_tool_calling_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# The agent autonomously decides if it needs to call the UC tool based on user input.
response = agent_executor.invoke({"input": "Check order history for Solaria
Properties."})
```

3. Utilizing Databricks Agent Bricks

Core Concept

Agent Bricks are out-of-the-box, managed Databricks solutions designed to accelerate GenAI application development. They offer declarative, high-level abstractions over complex architectures like

RAG pipelines, orchestration loops, and unstructured data parsing

[cite: <https://docs.databricks.com/aws/en/agents/custom-agents/build-agents>].

Available Bricks & When to Use Them

- **Information Extraction:** Transforms unstructured text (e.g., PDFs, invoices, contracts) into structured key insights (JSON) using a visual UI that relies on the `ai_extract` function. *When to use:* When standardizing unstructured data into tables for analysis, avoiding custom parsing code
[cite: <https://docs.databricks.com/aws/en/agents/agent-bricks/info-extraction>].
- **Knowledge Assistant:** Builds a domain-specific chatbot over your documents using an "Instructed Retriever" approach. *When to use:* For internal HR policies or product documentation QA where you want subject matter experts to provide natural language feedback directly via the UI to auto-tune the agent, bypassing traditional RAG limitations
[cite: <https://docs.databricks.com/aws/en/agents/agent-bricks/knowledge-assistant>].
- **Supervisor Agent (Supervisor API Beta):** Orchestrates multiple tools, Agent endpoints, UC functions, and MCP servers via a single POST request. Databricks natively runs the tool-selection and execution loop for you. *When to use:* When your agent uses only Databricks-hosted tools and you want to offload loop orchestration entirely to Databricks rather than maintaining custom Python state logic [cite: <https://docs.databricks.com/aws/en/agents/agent-framework/supervisor-api-app>].

Common Mistakes

Attempting to use the Supervisor API when client-side execution is required. The Supervisor API cannot mix Databricks-hosted tools with client-side function tools in the same request, and it does not support custom Python logic between tool calls (like conditional branching)

[cite: <https://docs.databricks.com/aws/en/agents/agent-framework/supervisor-api-app>].